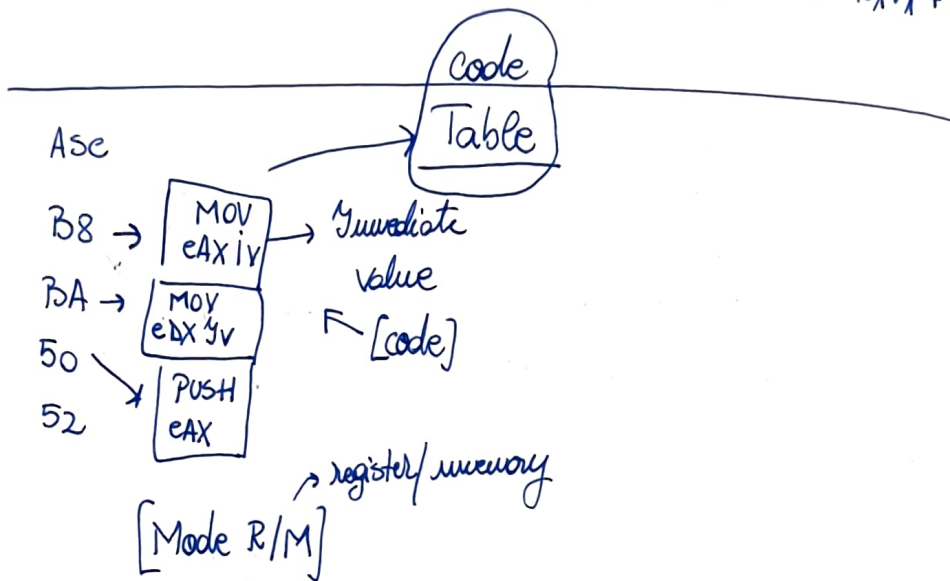


$$S \leq K \vee \begin{cases} S \neq \emptyset \\ (\forall) v_1, v_2 \in S, v_1 + v_2 \in S \\ (\forall) k \in K, (\forall) v \in S, k, v \in S \end{cases} \Rightarrow \begin{cases} S \neq \emptyset \\ (\forall) k_1, k_2 \in K, (\forall) v_1, v_2 \in S \\ k_1 v_1 + k_2 v_2 \in S \end{cases}$$



— it has to decide if it is a R/M

Mem

SIB $\rightarrow \sqrt{2 \text{ num formula}}$

How to express an offset?

index
 $a(7)$
 ↓
 base
 scale byte $\rightarrow 1$
 word $\rightarrow 2$
 dword $\rightarrow 4$
 qword $\rightarrow 8$

$[\] \rightarrow$ optional it can (diff from assembly)
 formula for computing addresses

$$\text{offset-address} = [\text{base}] + [\text{index} \times \text{scale}] + [\text{constant}]$$

$[\] \rightarrow$ dereferencing operator
 (the value from the address)
 (indications for the CPU)

$[\text{base}] \rightarrow$ any general register
 (which $\in$ executive unit ex: EAX, EBX, ...)

$[\text{index} \times \text{scale}]$ — n — no ESP $\rightarrow$ it can't be an index register (restrictions)
 ↓
 a constant: 1/2/4/8

[constant] → any constant value able to be determined/identified

offset - sp - form elements

at assembly time

! nu se incheadrea offset-formula

both of the operands

are expressed as register operands so the formula can't be applied

mov eax, ebx
mov eax, [ebx]

EAX
(offset-formula)

pointer variable expressed by register

[V] [ebx] → the content of EBX

memory addressing operands

00F748B9
EBX

in the memory

17 ← the value
00F748B9

mov eax, [ecx + 2 * edx + 7]

base index scale constant

mov eax, 23

mov eax, [23] → bug (olly debugger)

→ (correct!) but it's a bug in the debugger

mov eax, [ecx + 2 * esp + 7] → WRONG = SYNTAX ERROR
(ESP ≠ index register)

index reg. index order can be switched

! EXAMEN
10 DEC: mov eax, [ebx * 2] (no base; no constant)

mov eax, [ebx * 3] ⇔ mov eax [ebx + ebx * 2] so it's valid

↓ the scale can't be 3

or 9, 1248, ...

7 doesn't work

! mov eax, [ebx + ebx * 2 + ebx * 2]

! anyone can be present in our examples

SIB
Displacement } bytes

Scale Index Base

$$\begin{matrix} B & i & S \\ [base] & + & [index * scale] \end{matrix}$$

mov eax, [eax + 4 * ebx + 7]

mov eax, [v-7]

mov eax, [v] → address

mov eax, [7]

$$\text{mov eax, } \underbrace{\text{ebx} + 23}_{\text{reg}} \rightarrow \text{FORBIDDEN}$$

!!!
CS EIP
can be modified by
the programmer
with jumps

Mod R/M
B opcode

EIP

EIP → instruction pointer (column 1)
(offset of the executing program)

mov [EAX], ~~EBX~~ → CORRECT

mov [EAX], [EBX] → SYNTAX ERROR

$$\text{mov (eax), 23} \rightarrow \text{CORRECT}$$
 (only 1 may be stored in RAM memory)

$$\downarrow$$
 byte / word / ...

7 $\leftrightarrow$ `mov eax, [ebx*8 - ebx]` WRONG
 (you can't have a "-"
 you can only add)

`mov eax, [v]`

$\hookrightarrow$ memory address prepared (that's why the formula can work)

! if there isn't a syntax error $\rightarrow$ it obeys the offset-formula
 offset = how far off the beg

the content of a variable = variable
 the offset of a variable = constant det in ass. time

`mov eax [v]`
`jump mai-inc`

`mov eax, [0042 B7A9]`
`jump [0084]  $\downarrow$  (go 84 bytes below)`

The segment part
 SEG offset

INDIRECT ADDRESSING $[base] + [index * scale]$

! this formula is applied for computing offsets

DIRECT ADDRESSING $\rightarrow$ with the name of the variable

- complement false-dez
 - Repr -
 -
 -

2's complement. Discussions and examples

representation

interpretation

10010011

 $\underbrace{\quad}_9 \quad \underbrace{\quad}_3 \quad \underbrace{\quad}_4$

unsigned = 144

signed = - 2's complement of 10010011 = -109 !

base 16 = zipped ~~representation~~ notation

↳ also a representation (another version of base 2)

a) 1 0000 0000

Admissible representation intervals

 $N \text{ positions (bits)} \rightarrow 2^N \text{ values}$

Theory Exam	}	$[0 \dots 2^N - 1]$ - unsigned interpretation
		$[-2^{N-1}, 2^{N-1} - 1]$ - signed interpretation

- on 1 byte

 $[0, 255]$ - unsigned interpretation $[-128, 127]$ - signed interpretation

- on 1 word

 $[0, 65535]$ - u. i. $[-32768, 32767]$ - s. i.

a) $\begin{array}{r} 1\ 0000\ 0000 - \\ \underline{1001\ 0011} \\ 01101101 \end{array}$

$\underbrace{\hspace{1cm}}_6 \quad \underbrace{\hspace{1cm}}_{18} \quad h$

D

} 109

$$\begin{array}{r} \text{b) } 0110\ 1100 + \\ \underline{\hspace{1cm}} \\ 0110\ 1101 \end{array}$$

7 c) ~~binary repr. of 4 hex number~~ → the best ^{generation} binary code of the 2's complement of an initial value

d) fastest one

!!! The sum of the absolute values of the 2^i complementary binary configurations / representations is the cardinal of the set of values representable on that size.

$$256 - 147 = 109$$

! identify the key words in every question/definition
(dacă nu, restantă)

EXAM :

Which is the signed interpretation of

a) 1001 0011 (2) 90 110 1101

ii) 93h

→ 6409

ii) $147_{(10)} (C)$

c) $60h$
 $d + 14z$

c) none of the above

0110 1101

iii) $\rightarrow$ none of the above !

because in the question it says "interpretation", ~~so~~
~~the~~ and 147 is already an interpretation
 (it should've ~~said~~ representation*)
 been an

-109 is not the 2's complementary of 147

in practice we use that

-109 and 147 2's comp values

is not reciproc

? $\begin{pmatrix} 109 \\ -147 \end{pmatrix}$

a) $\overline{0xxx \dots}$ $\rightarrow$ m. int. the same + $\overline{abc}$
 $\rightarrow$ s. int.

$$[0, 255] \cap [-128, 127] = [0, 127]$$

b) 1 — Which is the representation ~~for~~ — for $\overline{-abc}$

c) $\overline{1xxx \dots}$ $\rightarrow$ m. i. + $\overline{abc}$
 $\rightarrow$ s. i. $\ominus$ 2's complement on the initial configuration

The values in signed?

-(2's compl of $\overline{1xxx \dots}$)

d) Which is the representation for $\ominus DEF$?

- the 2's complement of $\overline{1xxx \dots}$

(the answer is wrong)

we don't have a 147 representation!

$[-128, 127]$

→ it has to be something > byte.

CORRECT
ANSWER

the 2's complement of the unsigned extension

on an immediate superior size of the initial

$\overline{00 \dots 1xxx \dots}$ configuration

CORRECT ANSWER

0000 0000 1001 0011
↓

1111 1111 0110 1101 = FF6 8

2's complement

-147

g

Which is the binary representation that
makes complementary 109 and -147 (padded)

-109 the es

? Which is the minimum number of bits on which we can represent a number $[3]$

→ intervals admissible---

→ det N ? (from the ~~set~~, $[0, 2^N - 1]$)

$$1 \text{ bit} \rightarrow 2^1 = 2 \quad [0, 1]$$

$$2 \text{ bits} \rightarrow 2^2 = 4 \quad [0, 3] \text{ u.i.}$$

$$[-2, +1] \text{ s.i.}$$

ANS: 2 BITS (Value 11)

? ~~11~~ 11 $[3]$

$$3 \text{ bits} \rightarrow 2^3 \quad [0, 7]$$

$$[-4, 3]$$

ANS: 3 BITS (Value ~~11~~)

$$\begin{array}{c} 011 \\ \hline 101 \end{array} - 3 \quad (\text{in the unsigned is } 5)^{101_{(2)}}$$

ASE - Curs 6

10.11.2022

ASSEMBLY LANGUAGE BASICS

- main task assembly = generating bytes

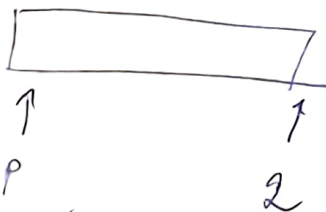
Pointer arithmetic

1. The only operation that can exist between 2 pointers is SUBTRACTION (because it makes sense)

\$ -

↓
the point where I am
at the current moment

- ! Any segment has its own location counter



$(q - p) \rightarrow$ the number of bytes between 2 pointers

2. Adding a constant to a pointer

$a[7] = *(a + 4)$ (the content of $(a + 7)$)

$p + 7$ (pass an array
constant as an index)

3) Subtracting a constant

$p-7$

→ What if it goes out of it?

→

in MASM

! $q + p$ → doesn't report a syntax error
↳ it doesn't mean addition of 2 pointers

$$a[7] = *(a+7) = *(7+a) = 7[a]$$

$p+7$ → POINTER DATA TYPE

$q-p$ → SCALAR VALUE

$$[\text{base}] + [\text{index} * \text{scale}] (+ [\text{const}])$$

V db 17

a dw 232, -17, 1xy'

b dd 12345678h

add EDI, [EBX+ECX*2+V-17] ✓

mov EBX, [EBX, ECX*2+a+b-7] ~~→ syntax error~~

(sub [EBX+ECX*2+a+b-7])

→ mov EBX, [EBX, ECX*2-V-7] - syntax error

ASC - Curs 14

~~17.11.2022~~

17.11.2022

Location counter ~~has the type pointer~~ is an integer number of type pointer

AND

$x \text{ AND } 0 = 0$	OR	$x \text{ OR } 0 = x$
$x \text{ AND } 1 = x$		$x \text{ OR } 1 = 1$
$x \text{ AND } x = x$		$x \text{ OR } x = x$
$x \text{ AND } \sim x = 0$		$x \text{ OR } \sim x = 1$

XOR

$x \text{ XOR } 0 = x$	for complementing	
$x \text{ XOR } 1 = \sim x$		
$x \text{ XOR } x = 0$		
$x \text{ XOR } \sim x = 1$		

10 dec

12 ex de initializare a unui registru cu 0

```
AND EAX, 0
SUB EAX, EAX
XOR EAX, EAX
MOV EAX, 0
shl EAX, ...
```

a db 17, -1, 'xyz'

b dw 40001, -128, '13'

mov eax, [a]

mov eax, a

mov eax, [a+17]

mov ebx, [b-a]

mov eax, [a+17]

mov eax, 17

ANOU

mov eax, -7

mov eax, 0

mov eax, ebx - syntax error

aa equ 2

mov ah, aa

mov ah, 127
ffffh

mov ax, 127

value ^ (value) fff...f

r db

a dw

b dd

push r stack ← offset(r)

push [r] ✗ (it has n't dimensions, which has to be 16/32 bits)

mov eax, [r] ✓

push [eax] ✗

push 15 ✓ push dword 15
(exception in masn)

pop [r] ✗ (you don't specify the size of the target)
pop r; 2=3 (stupidity) r is attribute
pop eax

pop [eax] ✗

pop 15 ✗

mov [r], 0 ✗ (doesn't know if it is a byte...)

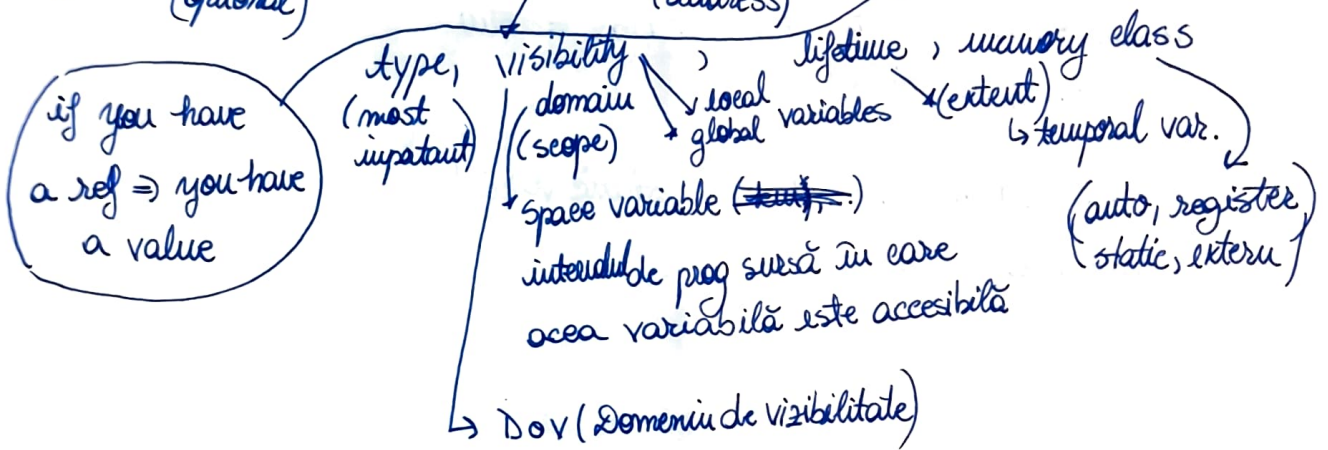
push [15] - should be ok, but

offset-size must representable
per 16/32 bits
per 8 bits

Variable (address, content) - NO

The concept of a variable in assembly language

Variable: (name, set of attributes, reference, value) - *unaduplu*
(optional) (address)



$\rightarrow$ the space allocated for a variable is cleared $\Rightarrow$ variable = dead

$\rightarrow$ variable = register (for efficiency)

$\rightarrow$ auto = memory class will adapt

• local $\rightarrow$ allocated in stack

• global $\rightarrow$ — — — global data segment

$\rightarrow$ heap $\rightarrow$ dynamic variables

$\rightarrow$ variabila pointer $\neq$ variabila dinamică

$\rightarrow$ p = new (----)

p-pointer to help managing the dynamic variable

$\rightarrow$ dynamic variable $\rightarrow$ accessed by a pointer

$\rightarrow$ free(p)

$\rightarrow$ Dynamic variables don't have names.

static, extern $\rightarrow$ very important

STATIC

import - export

(static) int f $\Rightarrow$ funcția derivă locală
 $\hookrightarrow$ face locală aceluși modul funcției respective

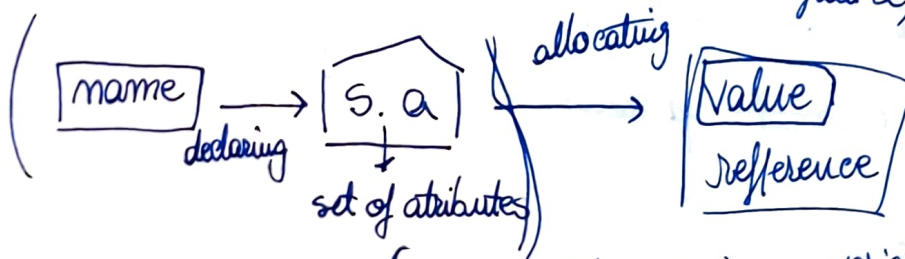
o var trăiește atâta timp cât DOV = activ
lifetime $\xrightarrow{\text{linked}}$ DOV

GDS $\rightarrow$ global variables + static variables
global data segment

EXTERN

linker - checks if ~~the~~ you have the same name for 2 variables
(the same variable for 2 modules)

definition = declaring its attributes + allocation
unique $\swarrow$ not unique $\searrow$ unique
(also a duplicate if you declare int a float a)



- formal parameter (int a, int b, int c) $\rightarrow$ variables that have a set of attr., memory but don't have a value

TIMES = directive that can also be applied
for instructions

EQU = constant def.

Segment data

i1 db 0,1,2,'xyz' ✓ (the assembler generates bytes 00 01 02)

db 300, 'F' + 3 ✓ (varying) $(300 = 12C) \neq (46) + 3 = 49$ va lui ult. 787776
 nu incupe intr-un octet
 octet din reprezentare indif de size

a2 TIMES 3 db 44h ✓ 44 44 44

a3 TIMES 11 db 5,1,3 ✓ 33 bytes (5 13 5 13 ...)

a4 db a2+1 (it doesn't fit a byte $\Rightarrow$ it will be a pointer)

a41 dw a2+1, 'bc'

a42 dw 1009h

a5 dd a2+1, 'bcd'

a6 TIMES 4 db '13'

a61 TIMES 4 dw '13'

a7 db a2

a8 dw a2

a9 dd a2

a10 dq a2 - 0000...0

a11 db [a2]

a12 dw [a2]

a13 dd [a2]

a14 dq [a2]

a15 dd eax

a16 dd [eax]

'1' '3' $\rightarrow$ concatenare de 2 octeti

'ab' 'a' 'b'

$\rightarrow$ generate the same $\nabla \nabla \nabla$ (10 dec)

'1' '3' $\rightarrow$ deja un word.

Syntax error!
 (cannot appear as ~~val~~ evaluable expressions
 at assembly time)

offsets on 16 bits or 32 bits
 but NEVER

on bytes $\Rightarrow$ syntax error

offset[a2]=8

from the start
 (0 1 2...)

→ exe. file → NOT the assembler
 generated by obj-file

→ generated by the assembler

→ The linker ~~editor~~ checks if a variable is defined twice

segment area = program area, ~~calculator~~ pre-calculator, ~~index~~ register
 mov ebx, [CS:C] → no syntax error

all 4 (CS, DS, ES, SS) start at the same address

FS, GS are not part of this ↑ (introduced on 32 bits)
 (3 rules CS, SS, DS)

mov eax, [ebx+esp] - SS ✓
 index ~~base~~

mov eax, [esp+ebx] - SS ✓

mov eax, [ebx+esp*2] - syntax error X (esp can't be an index)
 can be only base register

mov eax, [ebx+ebp*2] - DS ✓

mov eax, [ebx+ebp] - DS } the order

mov eax, [ebp+ebx] - SS

mov eax, [ebx*2+ebp] - SS

mov eax, [ebp*1+ebp] - SS (because EBP = base)

mov eax, [ebp*1+ebx] - DS (EBP = index, EBX = base)

mov eax, [ebx*1+ebp*1] - SS (EBX = index, EBP = base) } because we have 1

mov eax, [ebp*1+ebx*1] - DS (EBP = index, EBX = base)

mov eax, [ebp*1+ebx*1]

mov eax, [ebp*1+ebx*2] - SS (EBP = base (sets first the base, but when goes further and sees *2

mov eax, [ebp] - SS (EBP = base)

mov eax, [4*ebp] - DS (no base → 3 rule)

→ sets EBX = base)

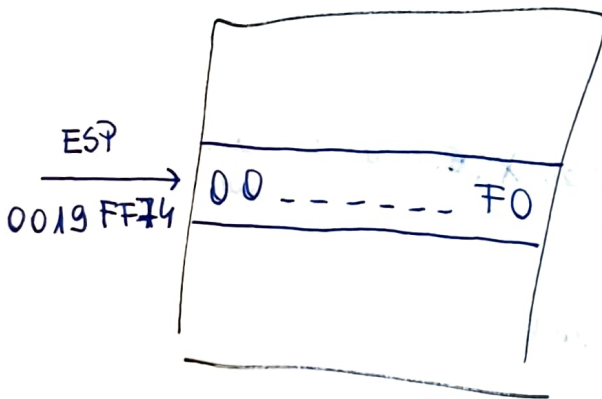
→ implicit operand - exists, but not seen

→ explicit operand - from input

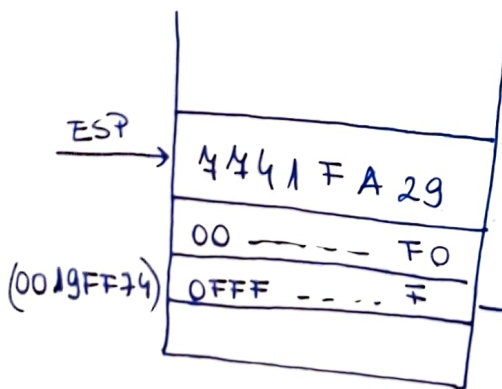
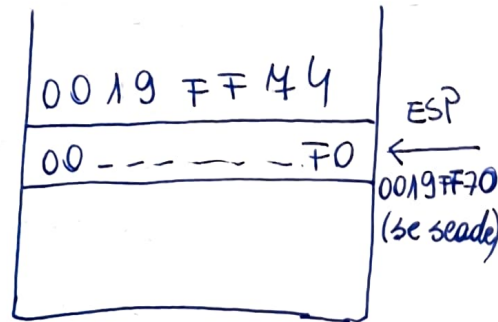
→ L-value

→ push source → dest = stack
stack = implicit operand

XL A ① → translation



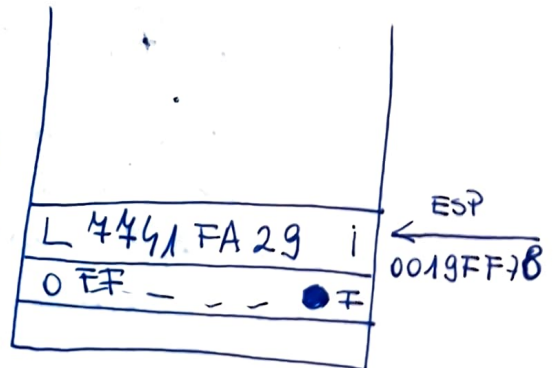
PUSH ESP



duword (correct)
(mean't push bytes)

POP ESP

syntax error
(size of operand
not specified)



→ evaluate operandul

→ actualizezi ESP-ul

→ actualizezi operandul

DF48ABFh (+ 'a' - 10)

AL = 7

↓ (XLAT) overwrites AL with the ascii code of AL (7)
(as an index)
14
relative to DS

ES XLAT → then is relative to ES (if not specified ⇒ DS)

register → address indirect

2 DATA SEGMENTS - ES, DS

LEA - load effective address

lea eax, [r] ⇒ mov eax, r (only for direct addressing operands!)

↳ for manipulating addresses in the case of indirect addressing

! lea ~~eax~~ eax, r (why not like this?) because the 2am formula wouldn't work

lea eax, [ebx + r - 6]

ce & eu reg → nu ~~re~~ me ce

1) operands interpretation (signed/unsigned)

? interpretation doesn't exist for a processor (only base 2)

only 2 situations
→ $\begin{array}{r} 1 \dots \\ 1 \dots \\ \hline 0 \dots \end{array}$ } u can't add 2 neg. numbers and obtain a pos. one
(OF = 1)
→ $\begin{array}{r} 0 \dots \\ 0 \dots \\ \hline 1 \end{array}$ } same (OF = 1)

→ overflow in multiplication:

m positions in base 10
n positions in base 10
result - (m+n)

! You never can't have overflow in multiplication

$$b * b \rightarrow b$$

$$w * w \rightarrow w$$

$$OF = CF = 0$$

$$b * b \rightarrow w$$

$$w * w \rightarrow dd$$

$$OF = CF = 1$$

no spec: case where instr. de majos setea $OF=0, CF=1$ (Tricky)

= mul
= int
= mul

(at multiplication

u can't have diff values at these flags)
only $OF=CF$

CORRECT - none of the above / no overflow

! division by 0 = OVERFLOW (because $\rightarrow \infty$)

mov [a], eax

movsx ah, [V] → we cannot fill → SYNTAX ERROR

movzx ax, [V] ; movzx ax, byte [V] (op size not specified)

movzx eax, [V] - syntax error (may be a byte/word, so the ass. doesn't know)

movsx eax, V - syntax error

! address is a direct not a memory address or a register or a constant it is seen as a constant (it should be between []) constant

mov eax, V

mov ~~eax~~ ax, V

segment data

Start

00401000h

...

Dest

00402008h

fictional subtraction

$$d < s \Rightarrow d - s < 0$$

! cmp and test are not comparing instructions (they only do fict. subtr.)
the comparison is made only when we reach the jumps
they just prepare the "area"

"N" - from RET N - remove N bytes from the stack (multiple of 4)
↳ std (apelantul)

! std - Manualie

Diunou:

mov eax, 89

jmp Maideparte:

Resd 1000h

Mai groupe:

jmp Diunou:

Maideparte:

mov ebx, 17
(add ebx, edx)

dec ecx
jnz ecx

Loop Diunou

→ short jump is out of range

→ max 127 in loop

→ loop → ~~error~~ (line Loop Diunou)

→ consider signed

→ long jump → not restricted to any number

→ short jump → restricted to 127

'x' → size of = 1 character

"x" → size of = 2 'x' '\0' string

a 6 dd "123" → exactly this order in memory
and completes with 0 till the
multiple of 4 (word, dword, etc)

'23' (≡) '2', '3'

mov → little endian thing

ASC-Curs 11

12.11.2022

Separate compilation means that in a program composed of n text files (source code files) anyone of them is compiled SEPARATELY at diff moments in time. The final executable file will be obtained as a final step linking together these binary .obj files using the link editor as a tool.

Using directives like `#include` does not mean sep. compilation and you will not obtain multinode programming.

! Exam *program curs bitdefender

Which are the shared resources between these 2 modules?

→ `loweaterware`

→ `guiFinal`

→ `eax, ebx`

! 3 possible categories of shared resources

→ the `call` instruction uses the stack

R. ~~static~~ symbols, `eax, ebx` and the stack (because `call` instr pushes the address on the stack and `ret...`)

→ `eax, ecx` - volatile resource

→ non-volatile resources should be saved

The implementation of calling a subroutine in computer science (file)

Steps: (~~calling~~ → subroutine call)

① Call phase → call code - C

② Entry phase → entry code - C

③ Exit phase → exit code - ASM

Call code ^{everything in asm is our responsibility}

- 1) Save volatile resources - EAX, ECX, EDX, eflags
- (2) Just to be careful
- 3) Passing parameters
- 4) Perform the ~~code~~ call

[printf] because the ~~code~~ ^{address}, it will take only the address

Exam

! phases, explain each, could be ~~an~~ ^{an} interview question

Entry code - most complex

a) Building the new stack frame

→ what involves to define a new

EBP →



stack frame?

EBP has to move (in the place of ESP), but ESP is overwritten so it has to be saved on the stack

push EBP

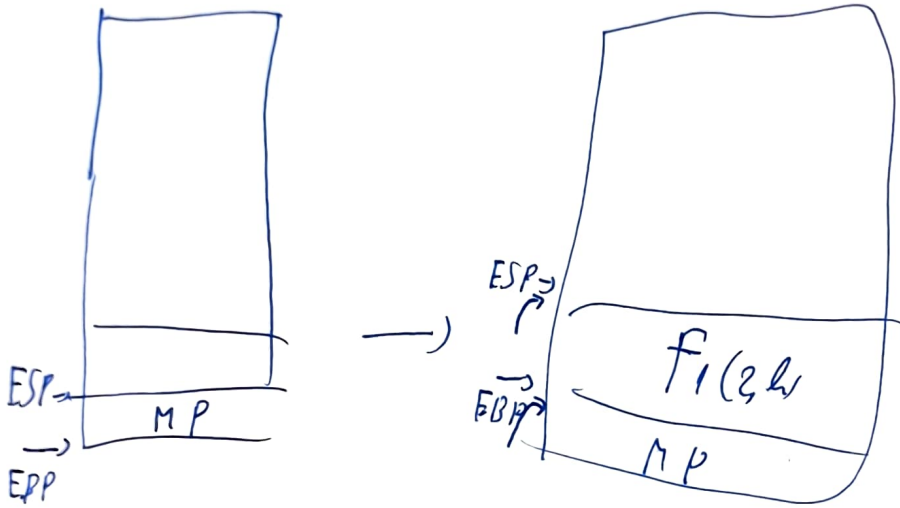
mov EBP, ESP

b) Reserving space for local variables
sub esp, nr-of-bytes

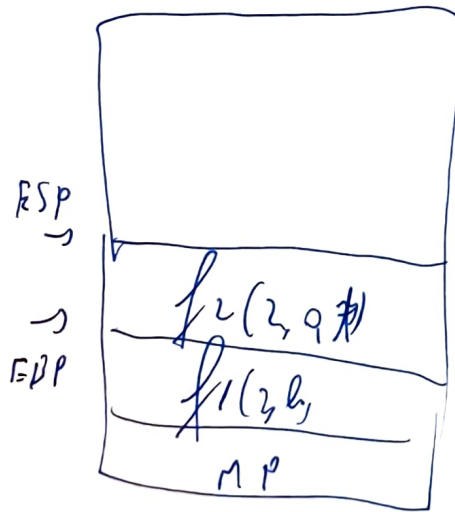
^{opposite} c) Save the non-volatile resources which know that will be affected

Exit code

- a) Restore the non volatile resources
- b) Release local variable space
- c) Deallocate the stack frame
- d) Returning step



EBP, ESP always
moves to the new function
on the stack!!!



- sign bit
- sign sub-seps.
- base 2 representation
- ↳ interpretation for a human being